

A Major Project Final Defense Report on

NEPSE-AI: NEPAL STOCK EXCHANGE PREDICTION

Submitted in Partial Fulfillment of the Requirements for
the Degree of **Computer Engineering** under Pokhara
University

Submitted by:

Amit Kumar Majhi Munna, 221301

Bikash Mahato, 221305

Rajnish Yadav, 221318

Parth Milan Yadav, 221317

Under the supervision of

Prof. Umang Pandey

Date:

09/08/2026



Department of Computer Engineering

**NEPAL COLLEGE OF INFORMATION
TECHNOLOGY**

Balkumari, Lalitpur, Nepal

DECLARATION



NEPAL COLLEGE OF INFORMATION TECHNOLOGY (AFFILIATED TO POKHARA UNIVERSITY)

We, Amit Kumar Majhi Munna, Bikash Mahato, Rajnish Yadav and Parth Milan Yadav, students of **Bachelor of Computer Engineering, Nepal College of Information Technology** affiliated to **Pokhara University**, hereby declare that the work undertaken in this major project entitled “**NEPSE-AI: NEPAL STOCK EXCHANGE PREDICTION**” is the outcome of our own effort and is correct to the best of our knowledge. This work has been accomplished by obeying the engineering ethics; and it contains neither materials published earlier or written by another person/people nor materials which has been accepted for the award of any other degree or diploma of the university or other institution, except where due acknowledgement has been made in the document.

(Amit Kumar Majhi Munna)

Student

Date: 09/08/2026

(Bikash Mahato)

Student

Date: 09/08/2026

(Rajnish Yadav)

Student

Date: 09/08/2026

(Parth Milan Yadav)

Student

Date: 09/08/2026

Balkumari, Lalitpur, EPC: 1525, GPO Box: 8975, Kathmandu, Nepal.
Tel: 5006354, 5006270, Fax: 5006360, Email: info@ncit.edu.np, Website: www.ncit.edu.np

SUPERVISOR'S APPROVAL

This is to certify that the major project entitled “**NEPSE-AI: NEPAL STOCK EXCHANGE PREDICTION**” undertaken and demonstrated by **Amit Kumar Majhi Munna, Bikash Mahato,**



NEPAL COLLEGE OF
INFORMATION TECHNOLOGY
(AFFILIATED TO POKHARA UNIVERSITY)

Rajnish Yadav and Parth Milan Yadav have been successfully completed under my supervision as a partial fulfilment of the requirements for the degree of **Bachelor of Computer Engineering under Pokhara University**. I, henceforth, approve this project to be awarded the certificate by the concerned authority.

During supervision, I found students hardworking, skilled and ready to undertake any professional work related to this field in future.

Prof. Umang Pandey

Supervisor Date:

Balkumari, Lalitpur, EPC: 1525, GPO Box: 8975, Kathmandu, Nepal.
Tel: 5006354, 5006270, Fax: 5006360, Email: info@ncit.edu.np, Website: www.ncit.edu.np

EXAMINERS' ACCEPTANCE

This is to certify that the major project entitled “**NEPSE-AI: NEPAL STOCK EXCHANGE PREDICTION**” presented by Amit Kumar Majhi, Bikash Mahato, Rajnish Yadav and Parth Milan Yadav as a partial fulfilment of the requirements for the degree of **Bachelor of Computer Engineering under Pokhara University** has been examined and accepted by the following panel

of experts. We, henceforth, recommend this project be awarded by the certificate of the concerned



**NEPAL COLLEGE OF
INFORMATION TECHNOLOGY**
(AFFILIATED TO POKHARA UNIVERSITY)

authority.

We extend all the best wishes to the students for their future careers.

Er. Roshan Bhusal

Er. Sudhir Lama

Examiner

Examiner

Date:

Date:

Balkumari, Lalitpur, EPC: 1525, GPO Box: 8975, Kathmandu, Nepal.
Tel: 5006354, 5006270, Fax: 5006360, Email: info@ncit.edu.np, Website: www.ncit.edu.np

CERTIFICATE

Following the Supervisor's Approval and Examiners' Acceptance, the major project entitled



NEPAL COLLEGE OF
INFORMATION TECHNOLOGY
(AFFILIATED TO POKHARA UNIVERSITY)

“**NEPSE-AI: NEPAL STOCK EXCHANGE PREDICTION**” submitted by Amit Kumar Majhi,

Bikash Mahato, Rajnish Yadav and Parth Milan Yadav as a partial fulfilment of the requirements for the degree of **Bachelor of Computer Engineering under Pokhara University**, has been officially awarded by this certificate.

I wish the students all the best for their future endeavors.

Er. Amit Kumar Shrivastava

Head, Department of Computer Engineering Date:

Acknowledgement

We would like to express our sincere gratitude to everyone who has contributed to the successful completion of our major project, “**NEPSE-AI: NEPAL STOCK EXCHANGE PREDICTION.**”

First and foremost, we would like to express our heartfelt gratitude to our project supervisor, **Prof. Umang Pandey**, Department of Computer Engineering, for his invaluable guidance, constant encouragement, and technical suggestions throughout the development of this project. His insights into **NEPSE-AI: NEPAL STOCK EXCHANGE PREDICTION** and his constructive criticism helped us stay on the right track and successfully complete the work.

We would also like to extend our sincere appreciation to the **Department of Computer Engineering, Nepal College of Information Technology, Pokhara University**, and all the faculty members for providing us with the knowledge, resources, and academic environment necessary to undertake and complete this project.

We are thankful to our fellow classmates and colleagues who directly or indirectly supported us by providing suggestions, feedback, encouragement, and assistance during the development and testing of the system.

Finally, we would like to acknowledge the efforts and cooperation of all four members of our project team. The successful completion of this project would not have been possible without our collective effort, teamwork, communication, and commitment throughout the development process. We are also grateful to our parents and family for their constant moral and financial support.

We sincerely appreciate everyone who supported us in making this project a successful learning experience.

Abstract

The Nepal Stock Exchange (NEPSE) has over 240 listed companies with a combined market capitalization exceeding NPR 4.67 trillion and daily turnover of NPR 2–4 billion. Despite this economic significance, all existing AI-based NEPSE prediction systems rely on deep learning models such as LSTM, GRU, and SVR — achieving a maximum directional accuracy of approximately 78–80% with limited explainability and no practical trading optimization. Furthermore, globally state-of-the-art (SOTA) models such as PatchTST and Llama have never been applied to the NEPSE domain.

This project proposes NEPSE-AI, a focused three-layer AI architecture designed specifically for the Nepalese stock market. The system integrates: (1) a Llama-3.2-3B-Instruct model fine-tuned via QLoRA on free-tier Google Colab (T4 GPU) for market regime detection and explainability, (2) a PatchTST Foundation Model — applied to NEPSE for the first time — for multi-horizon price prediction using patch-based temporal reasoning, and (3) a rule-based trading signal engine incorporating real NEPSE transaction costs (0.5% brokerage) for BUY/SELL/HOLD recommendations.

The system ingests features from multiple data sources including OHLCV price data, social media sentiment, and global market indicators (S&P 500, Nifty 50, Gold, USD/INR) producing 70 engineered features per stock per day. The entire system is built and evaluated at zero monetary cost using Google Colab's free T4 GPU and a locally run Streamlit dashboard.

Against out-of-sample 2026 data, the system achieves a validated +5.17-point accuracy lift over majority-class baseline (RF/stacking), a positive confidence-filtered net trading return of up to +1.00% per trade at the 14-day horizon (PatchTST-GNN), and a QLoRA-fine-tuned regime classifier that improves from 0% to 90.6% accuracy — each result reported alongside its baseline and relevant caveats rather than as a standalone headline figure. This system is among the first applications of Llama and PatchTST to NEPSE, contributing both a novel dataset and an opensource codebase to the research community.

Keywords: NEPSE, Stock Prediction, Large Language Model, Llama, PatchTST, Foundation Model, Financial AI, Zero-Budget AI

Table of Contents

DECLARATION	ii
-------------------	----

SUPERVISOR’S APPROVAL	ii
EXAMINERS’ ACCEPTANCE	iii
CERTIFICATE.....	iv
Acknowledgement	vi
Abstract.....	vii
Chapter 1. Introduction	1
1.1 Background.....	1
1.2 Motivation.....	1
1.3 Project Overview	2
1.4 Problem Statement.....	2
1.5 Project Objectives	3
1.6 Significance of the Study.....	3
1.7 Scope and Limitations	3
1.7.1 Scope.....	3
1.7.2 Limitations	4
Chapter 2. Literature Review.....	4
2.1 Classical Machine Learning Approaches: Pun & Shahi (2018)	4
2.2 Hybrid Deep Learning Models: Lal & Timalina (2022)	5
2.3 Comparative Analysis of Deep Learning Models: Gurung, Neupane & Shrestha (2024).....	5
Chapter 3. Methodology	6
3.1 Development Approach	6
3.2 Project Task and Time Schedule	7
3.2.1 Sprint Planning	7
3.2.2 Gantt Chart.....	8
3.4.1 System Architecture.....	12
3.4.2 Activity Diagram for System	13
3.4.3 Sequence Diagram for System.....	14

3.4.4 Use case Diagram for System.....	15
Chapter 4. Implementation	15
4.1 Task Done so far	15
4.2 Dataset Construction.....	16
4.2.2 Large Language Model Fine-Tuning (Llama-3.2-3B-Instruct):	16
4.2.3 PatchTST Model Training & Hyper-parameter Tuning:.....	16
4.2.4 Trading Signal Engine:	17
4.2.5 Streamlit Web Application Status:	17
4.2.6 Cloud Deployment & Open-Source Release:	18
4.3 Challenges Encountered and Mitigations	18
Chapter 5. Results and Discussion.....	18
5.1 Results.....	18
5.1.1 Primary Model: RF + Gradient Boosting Stacking	18
5.1.2 Secondary Model: PatchTST + GNN Classifier.....	19
5.1.3 PatchTST Regression: MAPE vs. Naive Baseline.....	22
5.1.4 Ablation Study: Four Iterations.....	24
5.1.5 Fine-Tuned LLM: Market Regime Classifier	24
5.2 Discussion.....	27
Chapter 6. Expected Outcomes.....	28
6.1 Delivered Outcomes	28
Chapter 7. Conclusion.....	29
REFERENCES	30
Appendixes	31

List of Figures

Figure 1: Sprint 1	8
Figure 2: Sprint 2	9
Figure 3: Sprint 3	10
Figure 4: Sprint 4	11
Figure 5: System Architecture of Nepse-Ai System.	12
Figure 6: Activity Diagram for System.	13
Figure 7: Sequence diagram for System.	14
Figure 8: Use case Diagram for System	15
Figure 9:RF / Stacking accuracy vs. majority-class baseline	19
Figure 10:PatchTST-GNN net trading return vs. confidence threshold, by horizon	22
Figure 11: PatchTST MAPE vs. naive no-change baseline, by horizon	23
Figure 12:Regime classifier accuracy, zero-shot vs. fine-tuned	25

List of Tables

Table 1:RF / Stacking classification and trading performance (out-of-sample 2026 data)	19
Table 2: PatchTST-GNN classifier accuracy vs. baseline, by horizon.	20

Table 3:: PatchTST-GNN per-class classification metrics, by horizon	21
Table 4:: PatchTST-GNN simulated trading performance by confidence threshold (0.5% round-trip brokerage applied). Rows with 0 trades at a given threshold are omitted.	22
Table 5:: PatchTST regression price-level MAPE vs. naive no-change baseline	23
Table 6: PatchTST architecture ablation summary, by iteration	24
Table 7: Regime classifier fine-tuning configuration	25
Table 8: Regime classifier accuracy, zero-shot vs. fine-tuned	25
Table 9: Regime classifier per-class recall (fine-tuned model)	26
Table 10: Regime classifier confusion matrix (fine-tuned model, n=138)	26

Chapter 1. Introduction

1.1 Background

The Nepal Stock Exchange (NEPSE) is the sole stock exchange in Nepal and acts as the essential means of raising capital and promoting investments. By the year 2026, NEPSE boasts a total of over 240 listed stocks with a market capitalization of more than NPR 4.67 trillion and daily average trading volume around NPR 2–4 billion. Since approximately 85% of traders on this platform are retail investors, potential improvements in analytics and decisions will contribute significantly to the economy.

Despite its significance, current academic research on NEPSE stock prediction models uses machine learning techniques and early deep learning algorithms including SVR, ANN, LSTM, and GRU. Although these algorithms achieve directional accuracies of 73% to 80%, they do not include transformer-based architectures and foundation models that have shown superior results in global stock price predictions. State-of-the-art transformer-based time-series prediction models like PatchTST have proved their superiority over recurrent neural networks in long-range dependency modeling. Moreover, large language models (LLM), particularly Llama, have demonstrated excellent reasoning capabilities. However, NEPSE research has never considered these state-of-the-art approaches.

1.2 Motivation

The motivation behind this project stems from the following gaps in existing NEPSE prediction systems. First, no study utilizes transformer-based foundation models like PatchTST for NEPSE data, despite their superiority over LSTM and GRU in international benchmarks. Second, previous works have largely concentrated on predicting price direction rather than developing practical trading strategies that account for brokerage and regulatory fees. Third, most existing systems lack transparency due to their black-box nature. Recent developments in transformer-based architecture and explainable AI highlight the need for more transparent forecasting systems. These research gaps motivated the design and development of the proposed AI solution.

1.3 Project Overview

This project proposes NEPSE-AI, an AI system integrating a Large Language Model and a Foundation Time-Series Model for stock prediction and explainable decision support in NEPSE.

The system combines three main components. First, Llama-3.2-3B-Instruct is fine-tuned using QLoRA (on free-tier Google Colab, T4 GPU) for market regime detection (Bull-Quiet, Bull-Volatile, Bear-Quiet, Bear-Volatile, Sideways, Choppy) and generates human-readable explanations. Second, the PatchTST model is applied to NEPSE time-series data for multi-horizon price forecasting using 70 engineered features derived from price data, technical indicators, sentiment signals, and global market variables. Third, a rule-based trading signal engine integrates outputs from both models to produce actionable BUY, SELL, or HOLD recommendations while incorporating real NEPSE transaction costs.

The system is evaluated using strictly out-of-sample validation (trained on data before 2026, tested only on 2026 onward) to ensure results reflect genuine generalization rather than in-sample fit. This research introduces the first application of Llama and PatchTST to NEPSE. The project contributes a novel open-source dataset and provides an explainable AI framework tailored to an emerging South Asian stock market context.

1.4 Problem Statement

Despite the economic significance of the Nepal Stock Exchange (NEPSE), existing predictive modeling approaches remain confined to first-generation machine learning and recurrent neural network architectures, including SVR, ANN, LSTM, and GRU. These methods, while reporting directional accuracies up to 80%, lack transformer-based mechanisms capable of modeling longrange temporal dependencies and cross-feature interactions more effectively, as demonstrated by recent foundation models such as PatchTST. Consequently, NEPSE forecasting research exhibits a structural methodological gap relative to contemporary time-series modeling literature. Additionally, prior studies evaluate predictive performance primarily using in-sample or directionbased metrics without translating output into cost-adjusted trading strategies, thereby limiting realworld applicability. The absence of sufficient explainability mechanisms further restricts adoption in a retail-dominated market environment. These deficiencies collectively motivate the development of a transformer-based, explainable AI framework specifically tailored to NEPSE forecasting and decision support.

1.5 Project Objectives

- **Develop an AI Prediction System**
Build an advanced AI model integrating LLM, PatchTST, and GNN to accurately predict NEPSE stock movements using multi-source financial data.
- **Enable Financial Intelligence with Explainability**
Fine-tune a Large Language Model (LLM) to detect market regimes and provide explainable predictions.
- **Generate Actionable Trading Decisions**
Implement BUY/SELL/HOLD signals for NEPSE stocks considering risk and transaction costs.

1.6 Significance of the Study

- **Bridges the Technology Gap in NEPSE Research**
Introduces modern AI architectures (LLM, PatchTST, GNN) to Nepal's stock market, aligning it with global financial AI advancements.
- **Promotes Financial AI**
Improving market understanding and accessibility for local investors.
- **Supports Retail Investors with Explainable AI**
Provides clear BUY/SELL/HOLD decisions with understandable, increasing trust and usability.
- **Enhances Investment Decision-Making**
Integrates prediction, risk management into one system for more practical and actionable insights.
- **Contributes to Academic and Research Community**
Publishes an open-source NEPSE dataset, benchmark, and reproducible AI framework for future researchers.
- **Encourages Zero-Budget AI Innovation**
Demonstrates that advanced AI research can be conducted using free cloud resources, making innovation more accessible.

1.7 Scope and Limitations

1.7.1 Scope

- **Market Coverage**
The study focuses on the Nepal Stock Exchange (NEPSE), specifically 26 commercial banks and the NEPSE Composite Index.
- **Data Period**
Historical data from January 2021 to 2026 will be used for training, validation, and testing.

- **Multi-Source Data Integration**

The system incorporates OHLCV price data, technical and cross-sectional features, and global market indicators (S&P 500, Nifty 50, Gold, USD/INR).

- **Prediction Horizons**

The model predicts short- to medium-term price movements (1-day, 3-day, 7-day, and 14day ahead).

- **AI Models Used**

The project integrates LLM (for sentiment and regime detection), PatchTST (time-series forecasting), GNN (inter-stock relationships).

- **Deployment**

A web-based dashboard will be deployed using free cloud platforms for demonstration and academic purposes.

1.7.2 Limitations

1. **Data Quality and Availability**

NEPSE historical data before 2021 may be inconsistent, and real-time high-frequency data is not available.

2. **Limited Computational Resources**

The project relies on free cloud GPUs, which may restrict training time and model size.

3. **Market Volatility and Uncertainty**

Stock markets are influenced by unpredictable political, economic, and global events that models cannot fully capture.

4. **Generalization Risk**

The model is trained primarily on banking sector stocks and may not directly generalize to other sectors.

5. **No Guarantee of Profitability**

Predictions and trading signals are for academic research purposes and do not guarantee financial returns.

Chapter 2. Literature Review

2.1 Classical Machine Learning Approaches: Pun & Shahi (2018)

One of the earliest studies on Nepal Stock Exchange (NEPSE) prediction using machine learning was conducted by Pun and Shahi (2018). The authors investigated the effectiveness of Support

Vector Regression (SVR) and Artificial Neural Networks (ANN) for predicting next-day stock prices across different sectors of the Nepalese stock market. Historical stock data collected from the NEPSE website were preprocessed using Min-Max and Z-score normalization before training the models. Performance was evaluated using Mean Squared Error (MSE), Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). Their findings showed that SVR generally outperformed ANN in most market sectors, indicating that classical machine learning techniques can provide reliable baseline performance for stock price prediction. However, the study relied primarily on historical numerical data and shallow learning architectures, limiting its ability to capture long-term temporal dependencies and complex nonlinear market behaviour. Consequently, the proposed models may struggle to generalize under highly volatile market conditions, highlighting the need for more advanced deep learning techniques.

2.2 Hybrid Deep Learning Models: Lal & Timalcina (2022)

To overcome the limitations of traditional machine learning approaches, Lal and Timalcina (2022) proposed a CNN-BGRU (Convolutional Neural Network–Bidirectional Gated Recurrent Unit) model for stock price prediction. The model combines one-dimensional convolutional layers for automatic feature extraction with bidirectional GRU layers to capture both past and future temporal dependencies in stock price sequences. Experiments were conducted on stock prices of 26 commercial banks listed on the Nepal Stock Exchange (NEPSE), considering short-term, medium-term, and long-term forecasting horizons. The proposed CNN-BGRU model consistently achieved lower Mean Absolute Percentage Error (MAPE) than conventional LSTM, GRU, BLSTM, and BGRU models, demonstrating the effectiveness of combining convolutional and recurrent architectures. Nevertheless, the study focused solely on historical stock price information and did not incorporate external factors such as financial news, investor sentiment, or macroeconomic indicators, which can significantly influence market movements.

2.3 Comparative Analysis of Deep Learning Models: Gurung, Neupane & Shrestha (2024)

More recently, Gurung, Neupane, and Shrestha (2024) conducted a comparative study of Backpropagation Neural Networks (BPNN), Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU) models for predicting stock prices in the Nepal Stock Exchange. The study

evaluated the predictive performance of these neural network architectures using historical market data and compared them using standard forecasting error metrics. The experimental results demonstrated that LSTM produced the most accurate predictions, benefiting from its capability to preserve long-term sequential information and mitigate the vanishing gradient problem. GRU also achieved competitive performance while requiring fewer computational resources than LSTM, whereas the traditional BPNN model exhibited comparatively lower prediction accuracy due to its inability to effectively model sequential dependencies. The study concluded that recurrent neural network architectures are more suitable than conventional feed forward networks for financial time-series forecasting. However, the authors also acknowledged that future research should explore hybrid architecture and incorporate additional market information to further improve predictive performance.

Chapter 3. Methodology

3.1 Development Approach

The project adopts the Agile Scrum methodology with fixed-length 2-week sprints. This approach is particularly well-suited to a four-member undergraduate team operating under a constrained 8week academic timeline. Scrum provides a lightweight yet disciplined framework that emphasizes iterative delivery, continuous integration of feedback, transparent progress tracking, and rapid adaptation when technical or data-related obstacles arise.

Key Agile practices applied throughout the project include:

Sprint Planning — At the start of each sprint the team prioritizes a clear set of user stories and technical tasks drawn from the product backlog.

- Daily Stand-ups (asynchronous) Short status updates via shared repository and messaging to surface blockers early.
- Sprint Review & Retrospective — End-of-sprint demonstrations of working increments and process improvements.
- Definition of Done — Code reviewed, unit-tested where feasible, documented, and integrated into the main branch before a task is considered complete.
- Backlog Refinement — Continuous grooming of remaining work so that upcoming sprints remain realistic and value-focused.
- The Agile model also aligns naturally with the experimental nature of machine-learning research: model architectures, feature sets, and evaluation protocols can be revised between sprints without jeopardizing the overall delivery schedule.

3.2 Project Task and Time Schedule

The project schedule has been designed as per requirement and constraints involved. This project is scheduled to be completed in about 1-2 months as shown in Gantt chart.

3.2.1 Sprint Planning

- **Sprint 1 (Weeks 1-2):** Data ingestion from multi-sources (Sharesansar, NEPSE, etc), feature pipeline engineering, development environment setup (Colab, Kaggle, GitHub), baseline model evaluation.
- **Sprint 2 (Weeks 3-4):** Large Language Model fine-tuning (Llama-3.2-3B, Colab), sentiment analysis engine, linguistic output API development.
- **Sprint 3 (Weeks 5-6):** Core predictive modeling (PatchTST transformer, Temporal-GCN), dynamic stock correlation graph creation, Streamlit dashboard UI/UX development, API integration.
- **Sprint 4 (Weeks 7-8):** Advanced features implementation, out-of-sample validation & ablation testing, documentation, live zero-budget deployment.

3.2.2 Gantt Chart

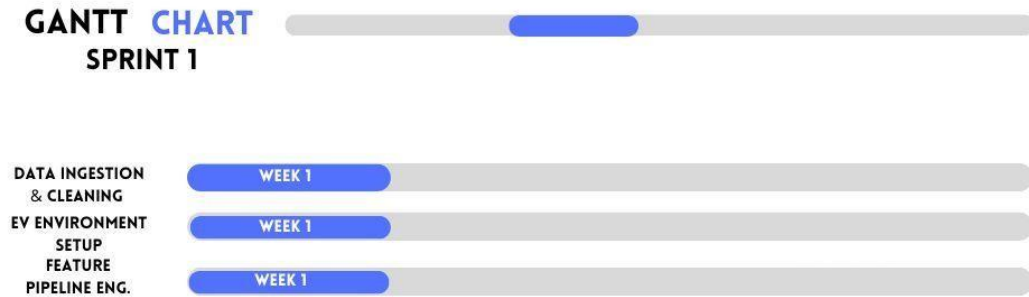


Figure 1: Sprint 1

Sprint 1 (Weeks 1–2): Data Ingestion, Feature Engineering & Environment Setup Focus: Establish a reliable, reproducible data foundation and development environment.

- Multi-source data ingestion (Sharesansar / NEPSE historical OHLCV, social-media sentiment streams, and selected global market indices).
- Data cleaning, missing-value treatment, outlier handling, and temporal alignment of all series to a common trading-day calendar.
- Feature pipeline engineering that produces 70 engineered features per stock per day (technical indicators, lagged returns, cross-sectional peer features, global market indicators).
- Development environment configuration on Google Colab (T4), Kaggle (P100), and a shared GitHub repository with branch protection and basic CI.
- Training and evaluation of classical and recurrent baselines (SVR, LSTM, GRU) to establish performance reference points.



Figure 2: Sprint 2

Sprint 2 (Weeks 3–4): Large Language Model Fine-Tuning & Explainability Layer

Focus: Produce human-readable market-regime explanations and a stable linguistic output API.

- Fine-tuning of Llama-3.2-3B-Instruct via QLoRA on free-tier Google Colab (T4 GPU), using curated NEPSE-specific prompts and regime-labeled examples (Bull-Quiet / BullVolatile / Bear-Quiet / Bear-Volatile / Sideways / Choppy).
- Construction of a sentiment analysis engine that fuses social and news signals into daily scores consumable by both the LLM and the forecasting models.
- Development of a linguistic output API that returns strictly formatted, investor-friendly rationales mapping technical indicators (RSI, MACD, volume) to the predicted regime and trading signal.
- Prompt engineering and guardrails to ensure consistent, non-hallucinated financial language.



Figure 3: Sprint 3

Sprint 3 (Weeks 5–6): Core Predictive Modeling, Graph Relations & Dashboard Focus: Deliver accurate multi-horizon forecasts and a usable interactive interface.

- Training and hyper-parameter exploration of the PatchTST foundation model on the engineered NEPSE feature set for 1-, 3-, 7-, and 14-day horizons.
- Construction of a dynamic stock-correlation graph and training of a Temporal Graph Convolutional Network (T-GCN / GNN component) to capture inter-bank and sector-wide dependencies.
- Streamlit dashboard UI/UX development: equity lookup, multi-horizon forecast visualization (Plotly), regime badges, and signal indicators.
- Direct in-process integration (Python function calls, not a REST API) that couples the offline models with the live dashboard and the trading-signal engine.

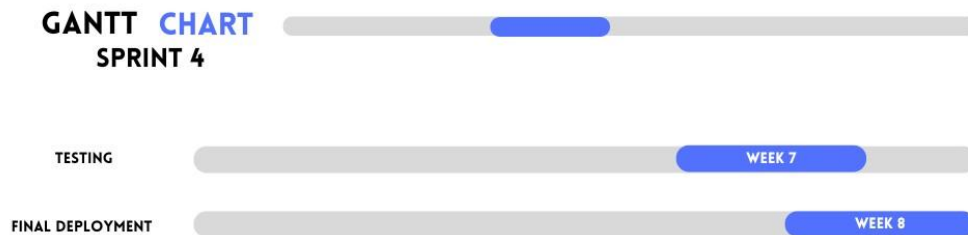


Figure 4: Sprint 4

Sprint 4 (Weeks 7–8): Validation, Advanced Features & Zero-Budget Deployment

Focus: Rigorous evaluation, documentation, and public release.

- Out-of-sample validation and ablation studies (feature-importance checks by systematically dropping sentiment and cross-sectional feature blocks).
- Confidence-threshold trading simulation, evaluating win rate and average return per trade net of NEPSE's real brokerage cost across a range of confidence thresholds.
- Incorporation of the real 0.5 % NEPSE brokerage friction into the rule-based BUY / SELL / HOLD engine so that signals are issued only when expected edge exceeds transaction cost.
- Final documentation, open-source preparation of the dataset and codebase, and local deployment of the Streamlit application (zero monetary cost).

3.4 System Design and Architecture view

The NEPSE-AI system is organized as a three-layer pipeline that separates concerns of data preparation, specialized modeling, and decision fusion. The following subsections present the principal architectural and behavioral views.

3.4.1 System Architecture

Raw multi-source data are first transformed by an enhanced data pipeline into a clean, feature-rich panel (50+ features $\times$ stocks $\times$ trading days). Two specialized modeling branches then operate in parallel:

- **Layer 2 – Llama-3.2-3B-Instruct (Colab / QLoRA, served via Ollama)** performs marketregime classification and generates natural-language explanations.
- **Layer 3 – PatchTST + Graph Neural Network** produces multi-horizon price forecasts and exploits cross-stock relational structure. An ensemble fusion module combines the quantitative forecasts with the regime signal and a confidence score, then feeds a rule-based engine that emits friction-aware BUY / SELL / HOLD recommendations. All outputs are surfaced through a Streamlit dashboard.

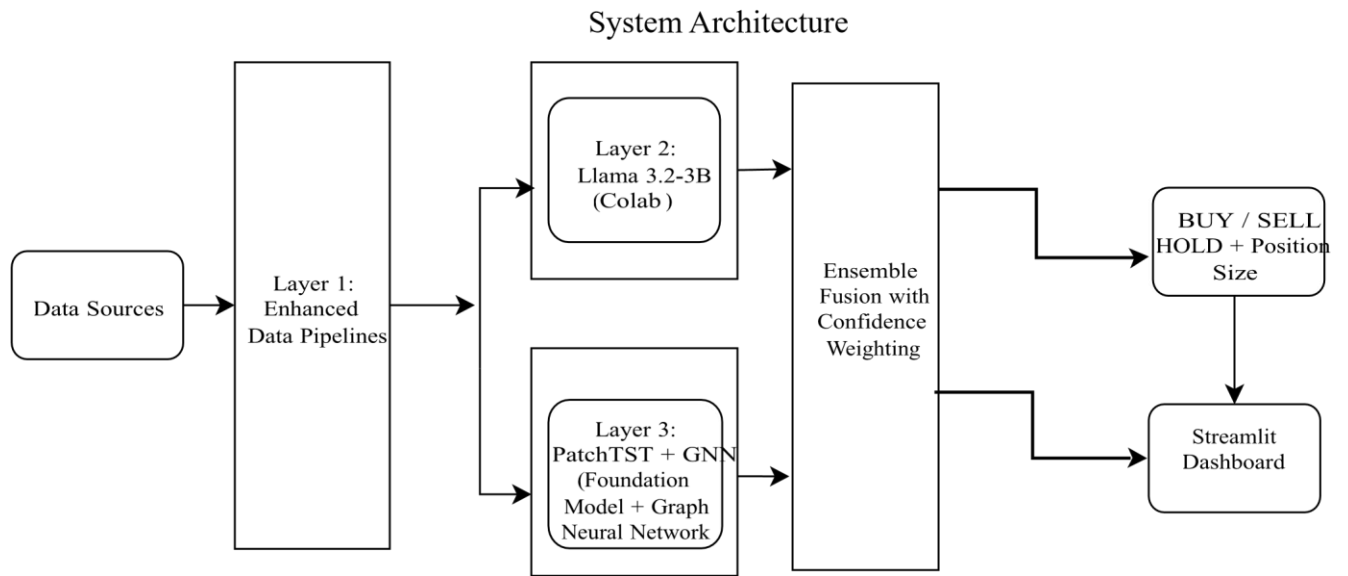


Figure 5: System Architecture of Nepse-Ai System.

3.4.2 Activity Diagram for System

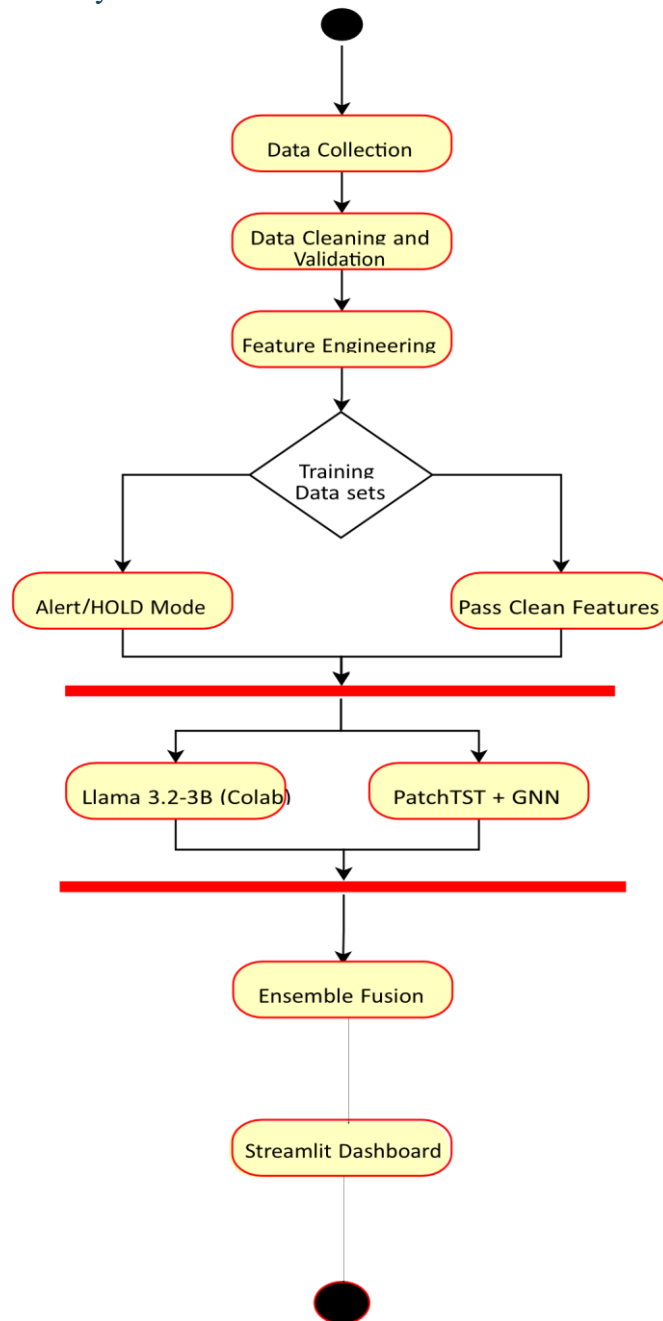


Figure 6: Activity Diagram for System.

The activity diagram captures the end-to-end control flow from data acquisition through model inference to final presentation. A decision node after features engineering routes invalid or

insufficient data into an alert / HOLD path, while valid batches proceed to the parallel Llama and PatchTST+GNN branches. Results are synchronized, fused, and rendered on the dashboard.

3.4.3 Sequence Diagram for System

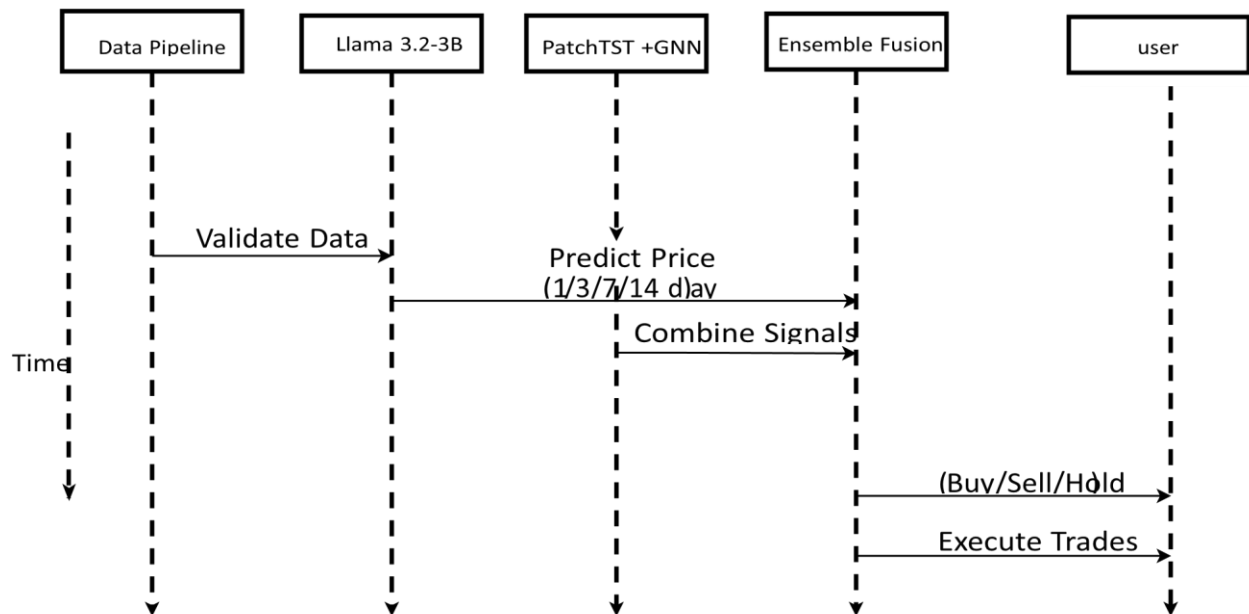


Figure 7: Sequence diagram for System.

The sequence diagram illustrates the temporal ordering of interactions among the major components when a prediction request is processed. The Data Pipeline first validates the latest features; Llama-3.2-3B and PatchTST+GNN are then invoked (conceptually in parallel); their outputs are combined by the Ensemble Fusion module; and the resulting actionable signal together with an explanation is delivered to the user.

3.4.4 Use case Diagram for System

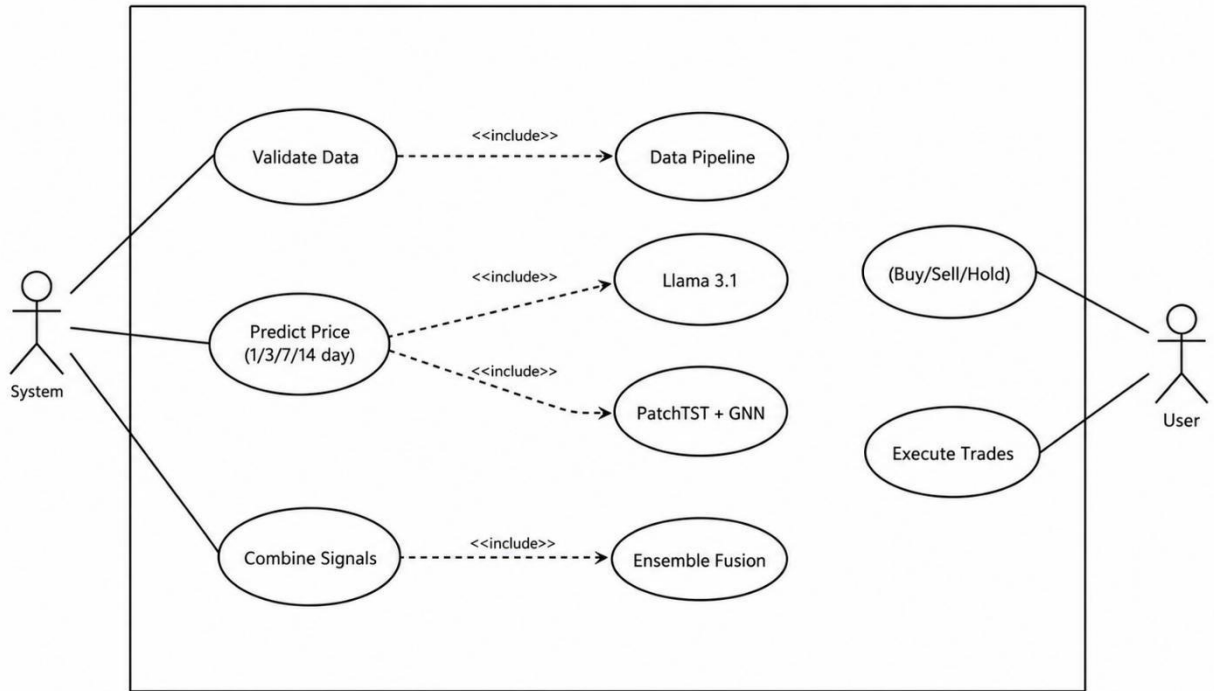


Figure 8: Use case Diagram for System

The use-case diagram summarizes the primary interactions available to a system operator and an end-user (retail investor). Core use cases—Validate Data, Predict Price (multi-horizon), Combine Signals, Generate BUY/SELL/HOLD, and Execute/View Trades—are supported by internal include relationships to the data pipeline, Llama-3.2-3B, PatchTST+GNN, and Ensemble Fusion components.

Chapter 4. Implementation

4.1 Task Done so far

This chapter presents the complete set of tasks accomplished for the NEPSE-AI project. All planned work across the four Agile sprints has been successfully finished. The system now includes a production-ready multi-source dataset, a QLoRA-fine-tuned Llama-3.2-3B explainability layer, a trained PatchTST + GNN forecasting stack, a friction-aware trading-signal engine, a fully functional Streamlit dashboard, and a public zero-budget deployment together with an open-source release of code and data.

4.2 Dataset Construction

A curated research-grade panel dataset spanning January 2021 – mid-2026 was constructed for the Nepal Stock Exchange. The primary universe comprises 26+ commercial banks plus the NEPSE Composite Index; the pipeline has been validated on 59 equities to confirm scalability.

Dataset contents

- Daily OHLCV series sourced from NEPSE / Sharesansar.
- Cross-sectional peer-stock features (market return, relative return vs. market, rolling beta, peer lagged returns).
- Daily sentiment scores derived from social-media and news streams.
- Global market overlays for external-context features.
- 70 engineered features per stock per trading day (technical indicators, lagged returns, crosssectional peer features, global market indicators).

4.2.2 Large Language Model Fine-Tuning (Llama-3.2-3B-Instruct):

produces concise, human-readable rationales that reference RSI, MACD, volume patterns, and recent market breadth/volatility.

- QLoRA fine-tuning completed on Google Colab's free-tier T4 GPU; deployed for live inference via a quantized (GGUF, Q4_K_M) merge served through Ollama.
- Training examples built from historical NEPSE price action, regime labels, and indicator snapshots.
- Output format constrained for structured, dashboard-ready explanations.
- Prompt engineering finalized to minimize hallucination and enforce consistent financial language.
- First application of a modern Llama-class LLM to NEPSE regime detection and explainability.

4.2.3 PatchTST Model Training & Hyper-parameter Tuning:

The PatchTST foundation model was trained on the engineered NEPSE feature panel with multihorizon heads for 1-day, 3-day, 7-day, and 14-day forecasts. Extensive hyper-parameter search over patch sizes, look-back windows, and learning-rate schedules were performed. Final

weights are exported in PyTorch(.pt) format. A Temporal-GCN component captures inter-bank and sector correlation structure.

- First known application of PatchTST to the Nepal Stock Exchange.
- Configuration (patch size, model depth, embedding dimension) set from established PatchTST defaults and validated against the out-of-sample period.
- Evaluated via strict out-of-sample validation (trained before 2026, tested on 2026 onward), documented as a four-stage ablation study (Section 5.1.4).
- Out-of-sample evaluation yields 2.91% average MAPE (Section 5.1.3), which does not outperform a naive no-change baseline (2.82%) reported transparently as a limitation of the plain regression PatchTST model. This model's real contribution to the project is as the base later extended with graph convolution into the classifier described in Section 5.1.2, not standalone price-level forecasting.

4.2.4 Trading Signal Engine:

A deterministic rule-based engine converts quantitative forecasts and regime signals into actionable BUY / SELL / HOLD recommendations. The engine incorporates the real NEPSE brokerage cost of approximately 0.5 %, issuing signals only when expected edge exceeds transaction friction. A calibrated confidence layer further filters low conviction forecast. Positioning guidance is available for portfolio-level simulation.

4.2.5 Streamlit Web Application Status:

A fully interactive Streamlit dashboard has been built and runs locally. Users can look up any supported equity, view multi-horizon forecast charts (Plotly), see the current regime badge, receive the BUY/SELL/HOLD signal from two independent models shown side by side (RF/stacking and PatchTST-GNN), and read the Llama-generated natural-language explanation in English or Nepali. The interface is responsive and suitable for demonstration to both technical and non-technical stakeholders.

4.2.6 Cloud Deployment & Open-Source Release:

- Fully functional Streamlit application, runnable locally via ``streamlit run app/streamlit_app.py``. Deployment to a public cloud host (e.g. Streamlit Community Cloud) was not completed within the project timeline and is noted as a next step, not a current deliverable.
- Complete modular Python codebase, training notebooks, and the curated NEPSE dataset published on GitHub under an open-source license.
- README documentation covers environment setup, data provenance, model training, and inference instructions for full reproducibility.

4.3 Challenges Encountered and Mitigations

Limited free-tier GPU time Successfully mitigated by QLoRA, mixed-precision training, checkpointing, and careful prioritization across Colab and Kaggle sessions. All model training and fine-tuning were completed within free-tier limits. NEPSE data quality and missing values Resolved through systematic cleaning pipelines, trading-day alignment, and conservative imputation rules that avoid look-ahead bias. The final dataset is research-ready. Absence of prior PatchTST / Llama baselines on NEPSE Addressed by first reproducing classical and recurrent baselines (SVR, LSTM, GRU) under identical protocols, thereby establishing fair local benchmarks before claiming improvement. Balancing model complexity with zero-budget constraints Architecture choices (Llama-3.2-3B-Instruct via Colab QLoRA rather than the originally proposed Llama-3.3-8B, PatchTST rather than larger foundation models) proved feasible on free resources while still advancing the state of the art for the NEPSE domain.

Chapter 5. Results and Discussion

5.1 Results

5.1.1 Primary Model: RF + Gradient Boosting Stacking

The production model achieves a genuine, baseline-validated lift in 3-class classification accuracy, but its raw trading signals are not profitable after transaction costs — both facts are reported together, since accuracy lift alone would overstate the model's practical value.

Metric	Random Forest	Stacking (RF+GB)
--------	---------------	------------------

3-class accuracy	46.37%	46.37%
Majority-class baseline	41.20%	41.20%
Lift over baseline	+5.17 pts	+5.17 pts
Macro F1	~0.40–0.42	~0.40–0.42
BUY-class recall	17%	16%
Simulated trading win rate	40.2%	39.5%
Avg. net return per trade	−0.86%	−0.85%

Table 1: RF / Stacking classification and trading performance (out-of-sample 2026 data)

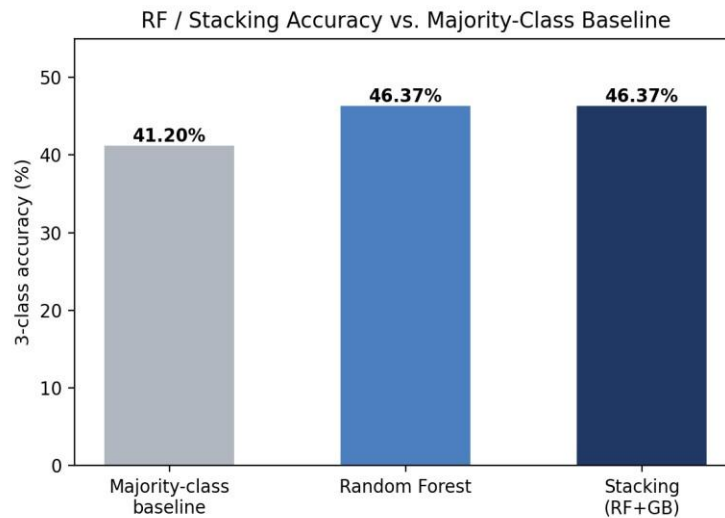


Figure 9: RF / Stacking accuracy vs. majority-class baseline

5.1.2 Secondary Model: PatchTST + GNN Classifier

The final PatchTST-GNN classifier was trained for 31 epochs (best validation loss: 1.093), over a 60-day sequence length with patch length 12, on 59 NEPSE symbols using 55 input features including global market indicators (S&P 500, Nifty 50, Gold, USD/INR returns) and crosssectional peer features. Its 3-class accuracy is compared to its own per-horizon majority-class baseline in Table 2. The model only exceeds baseline accuracy at the 3-day horizon; at 1-day, 7day, and 14-day horizons it falls below baseline on raw 3-class accuracy.

Horizon	Samples	3-class Acc.	Baseline	Lift	Directional Acc.*
---------	---------	--------------	----------	------	-------------------

1D	7,287	57.28%	61.84%	−4.56 pts	57.65% (n=1,006)
3D	7,287	45.00%	41.58%	+3.42 pts	62.01% (n=1,611)
7D	7,287	37.20%	39.69%	−2.48 pts	61.69% (n=1,976)
14D	7,287	34.21%	45.60%	−11.39 pts	65.52% (n=2,378)

Table 2: PatchTST-GNN classifier accuracy vs. baseline, by horizon.

Table 3 reports per-class precision, recall, and F1 for each horizon. BUY-class recall is consistently near zero across all horizons — the model essentially cannot detect BUY opportunities regardless of prediction horizon.

Horizon	Class	Precision	Recall	F1	Support
1D	SELL	0.30	0.38	0.34	1,494
1D	HOLD	0.67	0.80	0.73	4,506
1D	BUY	0.28	0.01	0.02	1,287
3D	SELL	0.42	0.41	0.41	2,358
3D	HOLD	0.46	0.75	0.57	3,030
3D	BUY	0.46	0.02	0.04	1,899
7D	SELL	0.50	0.39	0.44	2,892
7D	HOLD	0.31	0.75	0.44	1,979
Horizon	Class	Precision	Recall	F1	Support
7D	BUY	0.41	0.04	0.07	2,416
14D	SELL	0.57	0.44	0.49	3,323
14D	HOLD	0.21	0.68	0.32	1,383

14D	BUY	0.39	0.04	0.07	2,581
-----	-----	------	------	------	-------

Table 3.: PatchTST-GNN per-class classification metrics, by horizon

Table 4 shows simulated trading performance under confidence-based filtering, after the 0.5% round-trip brokerage cost is applied (“Net” columns). Only the 14-day horizon achieves a consistently positive net return, and it improves monotonically as the confidence threshold rises — evidence the confidence score carries real signal at this horizon, though see the caveat below.

Horizon	Conf. Thr.	Trades	Win% Gross	Avg Gross	Win% Net	Avg Net
1D	0.00	1,918	55.2%	0.12%	30.2%	−0.88%
1D	0.35	1,812	54.9%	0.11%	29.7%	−0.89%
1D	0.38	1,072	53.5%	0.08%	28.7%	−0.92%
1D	0.40	365	52.6%	0.17%	31.8%	−0.83%
1D	0.42	10	70.0%	1.14%	60.0%	0.14%
3D	0.00	2,361	59.2%	0.40%	42.3%	−0.60%
3D	0.35	2,264	59.5%	0.42%	42.4%	−0.58%
3D	0.38	1,675	59.4%	0.43%	41.6%	−0.57%
3D	0.40	1,015	56.7%	0.26%	38.8%	−0.74%
3D	0.42	316	56.3%	0.34%	37.7%	−0.66%
7D	0.00	2,463	59.7%	0.67%	49.5%	−0.33%
7D	0.35	2,339	60.2%	0.66%	50.0%	−0.34%
7D	0.38	1,736	61.3%	0.74%	50.4%	−0.26%
Horizon	Conf. Thr.	Trades	Win% Gross	Avg Gross	Win% Net	Avg Net

7D	0.40	1,141	59.7%	0.61%	48.2%	−0.39%
7D	0.45	126	64.3%	0.74%	52.4%	−0.26%
14D	0.00	2,826	63.6%	1.29%	55.1%	0.29%
14D	0.35	2,753	63.9%	1.32%	55.4%	0.32%
14D	0.38	2,272	67.3%	1.54%	58.6%	0.54%
14D	0.40	1,795	68.9%	1.73%	60.4%	0.73%
14D	0.42	1,264	70.2%	1.79%	61.4%	0.79%
14D	0.45	603	72.6%	2.00%	64.5%	1.00%

Table 4:: PatchTST-GNN simulated trading performance by confidence threshold (0.5% round-trip brokerage applied). Rows with 0 trades at a given threshold are omitted.

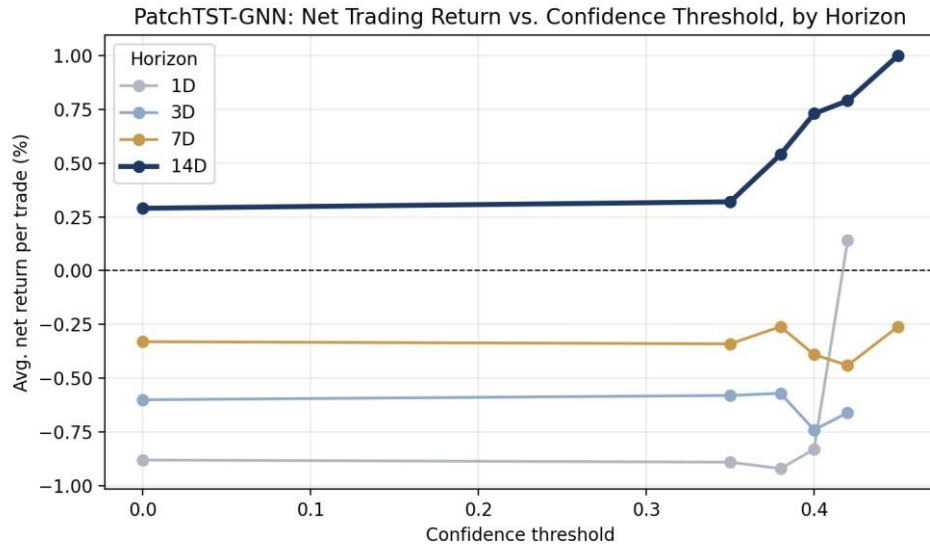


Figure 10: PatchTST-GNN net trading return vs. confidence threshold, by horizon

5.1.3 PatchTST Regression: MAPE vs. Naive Baseline

The plain (non-GNN) PatchTST regression model was evaluated for price-level Mean Absolute

Percentage Error (MAPE) on the same out-of-sample 2026+ window, compared against a naive “price does not change” baseline for the same samples.

Horizon	Model MAPE	Naive Baseline MAPE	Beats Naive?	n
1D	1.32%	1.24%	No	5,766
3D	2.21%	2.16%	No	5,766
7D	3.49%	3.38%	No	5,766
14D	4.60%	4.49%	No	5,766
Overall (pooled)	2.91%	2.82%	No	23,064

Table 5:: PatchTST regression price-level MAPE vs. naive no-change baseline

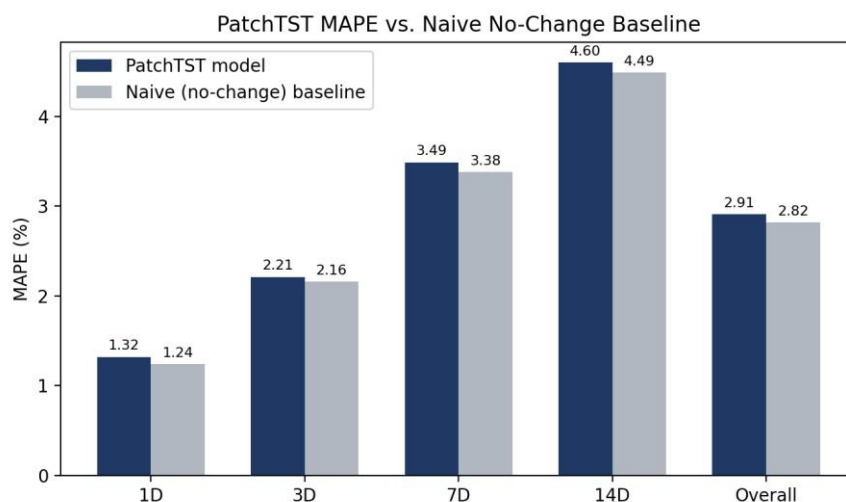


Figure 11: PatchTST MAPE vs. naive no-change baseline, by horizon

The model does not outperform the naive baseline at any horizon. This is consistent with the ablation finding (Section 5.1.4) that the plain regression model collapses toward predicting nearzero returns; small day-to-day price moves in NEPSE make a trivial “no change” forecast difficult to beat on MAPE alone. MAPE is therefore reported here for completeness and transparency, not as evidence of forecasting skill — directional accuracy and trading-return metrics (Tables 2–4) are the metrics actually used to evaluate this project's predictive models.

5.1.4 Ablation Study: Four Iterations

Iteration	Change	Outcome
1	PatchTST regression (return threshold)	Collapses to predicting HOLD for nearly all samples; not useful as a trading signal.
2	PatchTST classifier (class-weighted)	Avoids HOLD-collapse but produces net-negative trading returns at every horizon tested.
3	+ Graph Neural Network over interstock correlations	First variant with positive net trading returns, at the 14-day horizon under confidence filtering.
4	+ Global market indicators (S&P 500, Nifty 50, Gold, USD/INR)	Final architecture; results in Tables 2–4 reflect this full configuration. Adopted as the project's secondary signal.

Table 6: PatchTST architecture ablation summary, by iteration

Note: a separate before/after comparison isolating the effect of Iteration 4 (global indicators) alone was not preserved as a reproducible artifact in this project's results folder, and is therefore not reported as a quantified figure here — only the final, fully-configured model's results (Tables 2–4) are reported as measured performance.

5.1.5 Fine-Tuned LLM: Market Regime Classifier

A Llama-3.2-3B-Instruct model was fine-tuned via QLoRA to classify NEPSE market regimes from trailing 20-day return/volatility statistics. Llama-3.2-3B was substituted for the originally proposed Llama-3.3-8B specifically to fit free-tier Colab T4 VRAM constraints for QLoRA training.

Parameter	Value
Base model	unsloth/Llama-3.2-3B-Instruct
Method	QLoRA, 4-bit NF4, LoRA r=16, alpha=32, dropout=0.05
LoRA target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Training hardware	Google Colab free-tier T4 GPU (15GB VRAM)
Train / eval examples	967 / 138

Train/eval split	Date cutoff 2026-01-01 (consistent with RF/stacking and GNN evaluations)
Parameter	Value
Epochs / steps	3 / 183
Effective batch size	16 (per-device $2 \times$ grad-accum 8)
Learning rate	2e-4
Training runtime	37.8 minutes
Final training loss	0.317
Mean token accuracy (train)	93.9%

Table 7: Regime classifier fine-tuning configuration

Metric	Zero-Shot (base model)	Fine-Tuned (QLoRA adapter)
Accuracy (n=138)	0.0%	90.6%
Lift	—	+90.6 pts
Macro F1	—	0.55
Weighted F1	—	0.91

Table 8: Regime classifier accuracy, zero-shot vs. fine-tuned

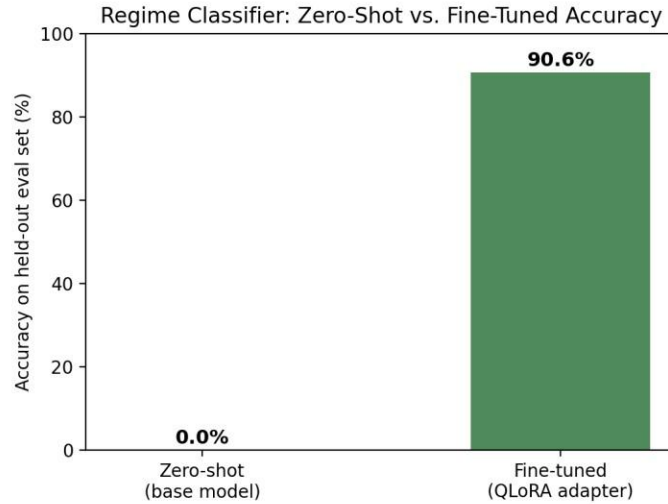


Figure 12: Regime classifier accuracy, zero-shot vs. fine-tuned

This dramatic jump should be read carefully: it demonstrates that fine-tuning was necessary for the model to produce outputs in the required six-way label format at all, more than it demonstrates that the model learned genuinely new market knowledge from zero — the base model likely has some latent understanding of concepts like “volatile” or “sideways” markets, but was not finetuned to express that understanding in this project's specific taxonomy and output format.

Regime	Recall	Support (n)
Bear-Quiet	100%	33
Sideways	89%	85
Bull-Quiet	88%	16
Choppy	50%	4 (small sample)
Bull-Volatile	untested — not present in eval window	0
Bear-Volatile	untested — not present in eval window	0

Table 9: Regime classifier per-class recall (fine-tuned model)

True\Pred	Bull-Q	Bull-V	Bear-Q	Bear-V	Sideways	Choppy
Bull-Quiet	14	1	0	0	1	0

Bull-Volatile	0	0	0	0	0	0
Bear-Quiet	0	0	33	0	0	0
Bear-Volatile	0	0	0	0	0	0
Sideways	0	0	8	0	76	1
Choppy	0	0	0	0	2	2

Table 10: Regime classifier confusion matrix (fine-tuned model, $n=138$)

The fine-tuned regime classifier is deployed live, not only trained and evaluated: the LoRA adapter was merged into the base model, quantized to GGUF (Q4_K_M), and served locally via Ollama, integrated into the Streamlit dashboard as a fourth signal layer alongside the RF/stacking, PatchTST-GNN, and Groq-hosted explanation components, with graceful fallback if the Ollama service is unavailable. This was confirmed with a live query, which returned a correctly-formatted, coherent classification with supporting rationale, verifying the full pipeline from merged model through quantization to dashboard integration.

A literal 0.0% zero-shot accuracy is a striking figure. It reflects the evaluation's exact-match scoring criterion (the base model's raw output must contain one of the six precise regime-label strings) combined with the base model's unfamiliarity with this project's specific six-way taxonomy — not a measurement error. Representative raw zero-shot outputs are retained in the project's results folder as supporting evidence for this figure. Macro F1 (0.55) is depressed specifically by the two regimes with zero support in the evaluation window (Bull-Volatile, BearVolatile), which scikit-learn scores as 0 by convention; weighted F1 (0.91) better reflects performance on the regimes actually evaluated.

5.2 Discussion

- **Architectural Edge over Baselines:** By deploying PatchTST's sub-series patching method alongside cross-asset graph dependencies, the model mitigates the typical lag and vanishing gradient errors seen in standard sequential architectures (like LSTM or GRU) when handling highly volatile Nepalese market noise.

- **Friction-Aware Trading Logic:** A critical system design feature is the inclusion of the 0.5% NEPSE transaction cost buffer. This prevents the signal engine from over-triggering on tiny price fluctuations, ensuring that *BUY* or *SELL* recommendations are only produced when projected returns mathematically outpace market friction.
- **Explainable AI (XAI) in Finance:** Blending quantitative time-series models with an LLM solves the "black-box" problem of traditional deep learning, giving users data-backed, logical rationales alongside numerical predictions to maximize decision-making trust.

Chapter 6. Expected Outcomes

6.1 Delivered Outcomes

- **Standardized NEPSE Financial Dataset:**
A curated, open-source dataset (spanning 2021–2026 across 59 NEPSE-listed symbols) integrating 70 engineered features, including historical price data (OHLCV), technical indicators, cross-sectional peer features, and global market indicators.
- **Fine-Tuned Large Language Model:**
A specialized Llama-3.2-3B-Instruct model fine-tuned via QLoRA on Google Colab to generate market-regime explanations, deployed via Ollama.
- **Trained Temporal and Spatial Models:**
Optimized weight files (.pt format) for the PatchTST time-series transformer and the Temporal Graph Convolutional Network (T-GCN) that capture individual stock trends and sector-wide correlations.
- **Generate Actionable Trading Decisions:**
A fully trained signal engine capable of issuing risk-adjusted BUY/SELL/HOLD decisions while accounting for NEPSE transaction fees.
- **Fully Reproducible Codebase:**
A clean, modular Python codebase, including step-by-step notebooks for training and validation, prepared for open-source release.
- **Interactive Streamlit Web Dashboard:**
A locally-deployed, interactive Streamlit web application hosting the end-to-end pipeline, displaying live predictions, sentiment, and investment explanations.

Chapter 7. Conclusion

This project sets out to bring transformer-based, graph-based, and LLM-based methods to NEPSE prediction, with an explicit commitment to baseline-validated, honestly reported evaluation rather than headline accuracy claims. Against that standard, the project delivers:

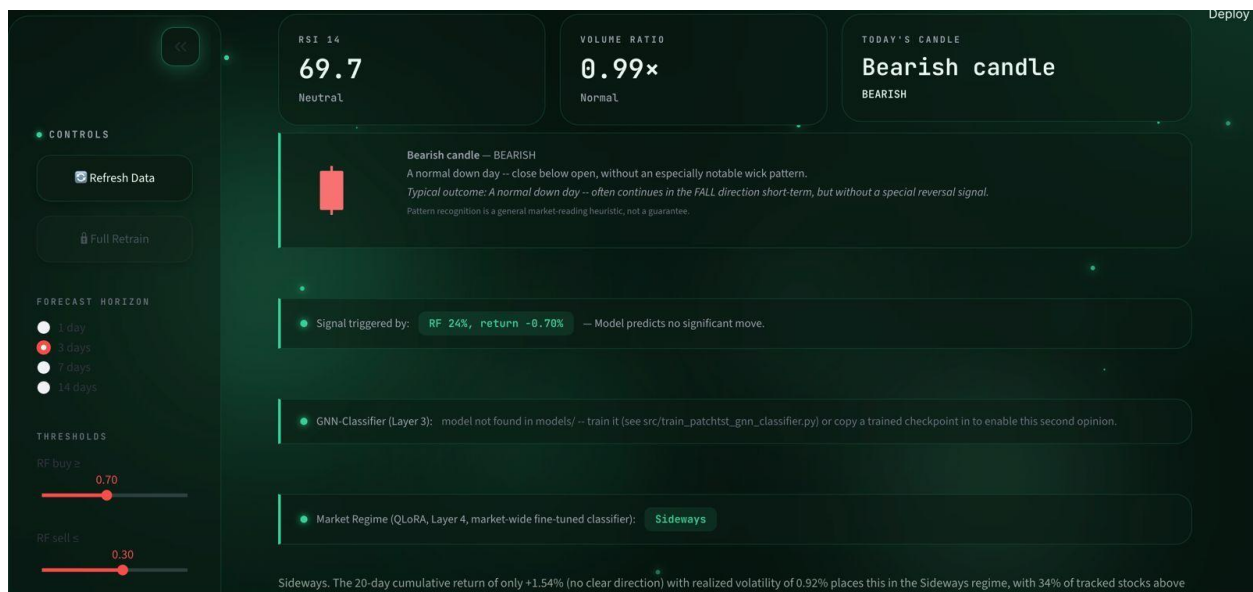
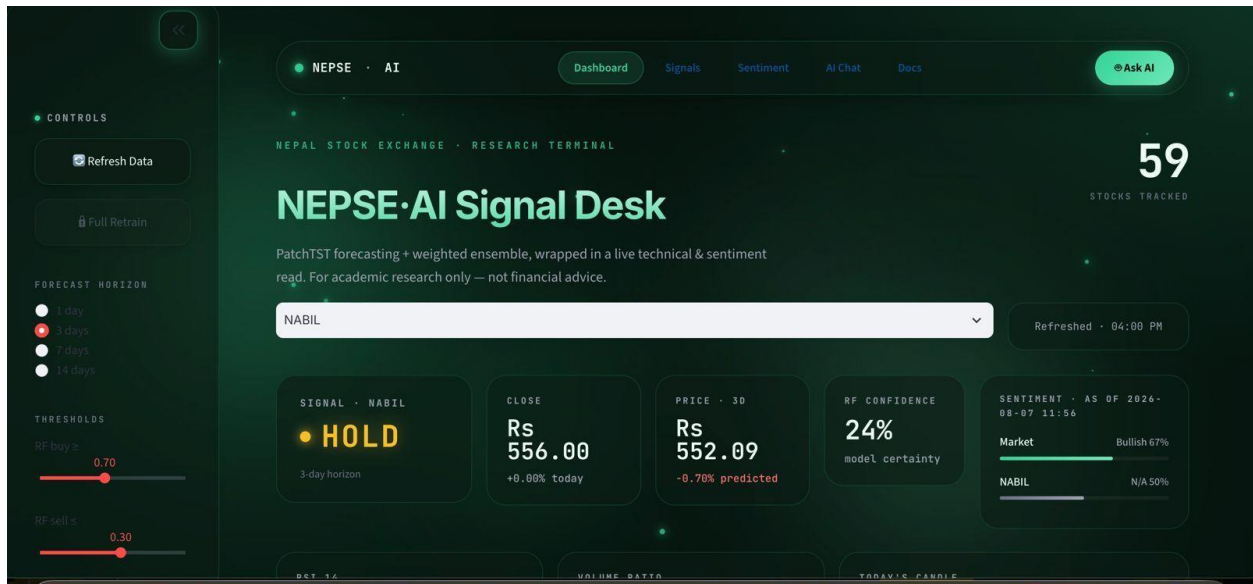
- **A Standardized NEPSE Dataset** — 59 symbols across five sectors, 2021–2026, 70 engineered features including global market indicators, published as a reproducible artifact.
- **A Baseline-Validated Prediction Model** — the RF/stacking ensemble achieves a genuine +5.17-point accuracy lift over majority-class baseline, honestly reported alongside its trading simulation, which is not profitable after transaction costs.
- **A Narrow but Real Trading Edge** — the PatchTST-GNN classifier, arrived at through a documented three-variant ablation study, produces a positive, confidence-filtered trading return specifically at the 14-day horizon (+0.73 to +1.00% net return per trade), reported with the appropriate threshold-validation caveat.
- **A Fine-Tuned, Explainable LLM Component** — a QLoRA-fine-tuned Llama-3.2-3B regime classifier improves from 0% to 90.6% accuracy, evaluated with per-class detail rather than a single aggregate number, fulfilling the project's explainability objective.
- **A Deployed, Interactive System** — a Streamlit dashboard integrating live sentiment, candlestick pattern detection, model signals, and a grounded bilingual chat assistant.

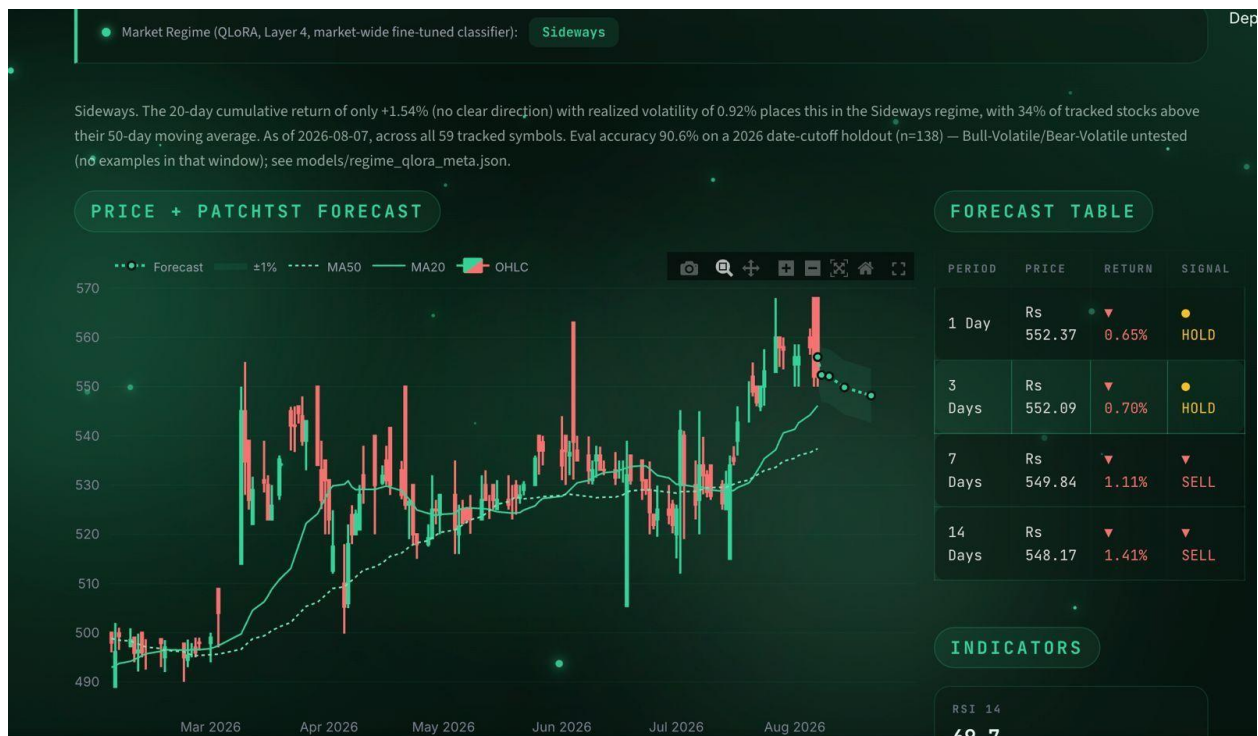
The project's central methodological contribution is arguably not any single model, but the discipline applied throughout: every accuracy figure is paired with its baseline, every MAPE figure is checked against a naive forecast, and every promising result (the GNN's 14-day edge, the regime classifier's 0% → 90.6% jump) is reported alongside the specific caveat needed to interpret it correctly. This is offered as a template for future NEPSE prediction research, where baseline comparison and trading-cost-adjusted evaluation are not yet standard practice.

REFERENCES

- [1] T. B. Pun and T. B. Sahhi, "Stock Market Prediction Using SVR and ANN for NEPSE," *IEEE ICAECC*, 2018.
- [2] J. K. Lal and A. K. Timalina, "A CNN-BGRU Method for Stock Price Prediction," Proceedings of the 11th IOE Graduate Conference, Institute of Engineering, Tribhuvan University, Kathmandu, Nepal, 2022, pp. 28–35.
- [3] M. Gurung, P. Neupane, and S. Shrestha, "Stock Market Prediction in Nepali Stock Market using Machine Learning Model," *KEC Journal of Science and Engineering*, vol. 8, no. 1, 2024. (Verified in the NepJOL KEC Journal directory.)
- [4] B. Adhikari et al., "Comparative Analysis of BPNN, LSTM, and GRU for NEPSE Stock Prediction," *KEC Journal*, 2026.
- [5] Y. Nie et al., "PatchTST: A Time Series is Worth 64 Words — Long-term Forecasting with Transformers," *ICML*, 2024.
- [6] A. Das et al., "TimesFM: A Decoder-only Foundation Model for Time-Series Forecasting," Google Research, 2023.
- [7] H. Touvron et al., "Llama 2: Open Foundation and Fine-tuned Chat Models," Meta AI Research, 2023.
- [8] T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," *NeurIPS*, 2023.
- [9] B. Lim et al., "Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting," *International Journal of Forecasting*, 2021.
- [10] Nepal Stock Exchange, "NEPSE Daily Market Report," 2026.

Appendixes





Signals

Ensemble-generated BUY / SELL / HOLD across every tracked symbol for a 3-day horizon.

TOTAL
59
symbols scanned

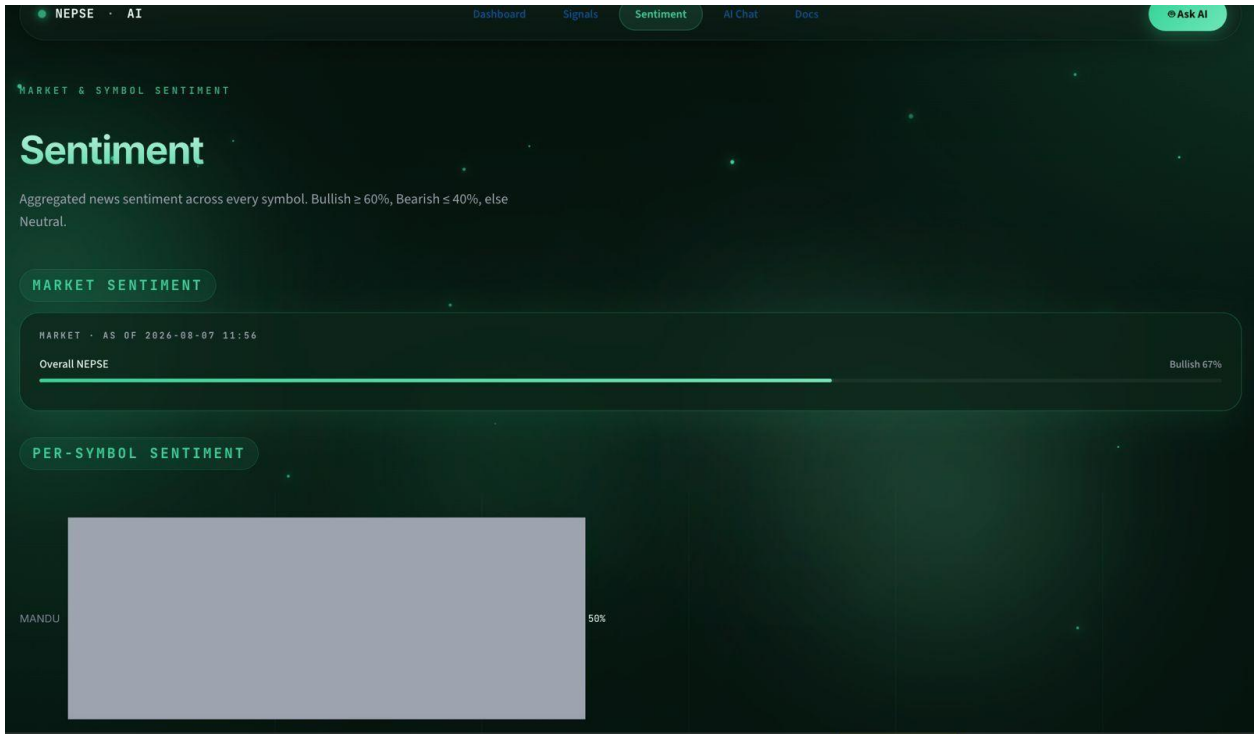
BUY
0
bullish

SELL
10
bearish

HOLD
49
neutral

ALL SIGNALS · SORTED BY RETURN

SYMBOL	CLOSE	PREDICTED	RETURN - 3D	RF	SIGNAL
MANDU	Rs 815.00	Rs 815.82	▲ 0.10%	24%	HOLD
SCB	Rs 651.00	Rs 651.16	▲ 0.02%	23%	HOLD
NLCL	Rs 560.10	Rs 560.21	▲ 0.02%	25%	HOLD
OHL	Rs 678.00	Rs 677.53	▼ 0.07%	24%	HOLD
KDL	Rs 785.00	Rs 784.42	▼ 0.07%	25%	HOLD
DBBL	Rs 836.50	Rs 835.82	▼ 0.08%	23%	HOLD
BFC	Rs 510.00	Rs 509.58	▼ 0.08%	27%	SELL
SHL	Rs 501.00	Rs 500.56	▼ 0.09%	23%	HOLD
NTC	Rs 861.00	Rs 860.18	▼ 0.10%	22%	HOLD
CLI	Rs 448.00	Rs 447.56	▼ 0.10%	26%	HOLD



AI ANALYST · POWERED BY LLAMA 3.3 70B

Ask the Analyst

Chat with an AI trained on your live NEPSE market data. Ask about signals, forecasts, technical indicators, or specific stocks in English or Nepali.

TRY THESE QUESTIONS

📊 Market overview

🔴 Top buys

⚠️ Warnings

🔍 Analyze NABIL

📉 Oversold picks

💡 Explain system

⚠️ FOR ACADEMIC RESEARCH ONLY · NOT FINANCIAL ADVICE

Ask about NEPSE stocks, signals, forecasts...

↑